GoreeCloud Memos Release and Rollback

- **Document type:** Engineering release and recovery procedure
- **Owner:** GoreeCloud
- **Status:** Active Development guidance; not a release authorization
- **Revision verified:** October 23, 2026
- **Canonical Markdown source:** `docs/RELEASE-AND-ROLLBACK.md`
- **Derived portable export:** `docs/RELEASE-AND-ROLLBACK.pdf`

Current boundary

GoreeCloud Memos is in Forge / Development. A green pull request or merged `main` revision is not a release, deployment, Production Acceptance result, Seal qualification, or Anchor qualification.

The repository currently has exact-revision validation and release-versioning metadata, but there is no active GoreeCloud release/deployment workflow that authorizes production publication from this repository. `release-please-config.json` is preparatory version/changelog configuration only; it is not release authority.

Production release remains blocked until the applicable GoreeCloud lifecycle, security, privacy, GC UI migration/conformance, accessibility, recovery, platform-integration, artifact-signing/provenance, deployment, and production-acceptance gates are satisfied.

Development-only update visibility

The Memos web client has an opt-in Development update notice to make a tested Development rollout recognizable on both the sign-in screen and the signed-in app. Default builds do not display it. A Development build may enable it **at frontend compile time only**:

```
VITE_GOREECLOUD_DEVELOPMENT_PREVIEW=true VITE_GOREECLOUD_SOURCE_REVISION=<exact-40-character-commit-sha> pnpm release
```

The notice reads the declared lifecycle stage and qualification state from `goreecloud.platform.yaml` at frontend build time, displaying **source metadata** rather than verified live runtime status. The build fails if either required manifest field is absent, duplicated, or unsupported. The current manifest declares **Forge** and **not-qualified**; the display also explicitly says runtime qualification is not verified. A reviewed source lifecycle transition must update the authoritative manifest and trigger a fresh build, exact-head validation, rendered acceptance, and deployment-source parity checks before any new status is communicated as runtime fact. The notice uses a neutral informational icon (not a success or release certification), may show an abbreviated valid commit, and provides a concise Development feature preview. It is collapsible, contains no user content, and does not authorize production publication. The flag must not be set for general-release/production builds. Verify the served hashed entrypoint asset against the exact compiled distribution, plus the rendered notice and revision on the isolated Development URL. Keep existing private data, backup, rollback, and provider gates intact.

State model

Keep these states separate:

1. working revision;
2. merge candidate;
3. merged revision on protected `main`;
4. release candidate;
5. immutable release/tag and exact artifacts;
6. deployment into a defined environment;
7. production acceptance;

8. lifecycle promotion when independently eligible.

No earlier state silently proves a later state.

Source merge gate

Before merging a material change:

- use current protected `main` as the base;
- identify the exact candidate head;
- review scope, dependencies, privacy/security impact, migration impact, documentation, and rollback considerations;
- require all applicable provider-enforced and GoreeCloud exact-head validation checks to pass;
- resolve required review threads and specialized review;
- treat any changed head as a new candidate; and
- merge only through repository protection.

The required validation jobs are documented in `.github/CONTRIBUTING.md` and `.github/workflows/goreecloud-validate.yml`.

After merge, verify the exact `main` revision and its post-merge checks. This completes source integration only.

Release-candidate gate

Before designating an exact merged revision as a release candidate, evidence must cover the applicable product risks, including:

- security and privacy acceptance;
- rendered GC UI migration/accessibility and representative browser/form-factor acceptance;
- compatibility and database-upgrade behavior;
- representative performance evidence and governed budgets;
- full recovery prerequisites appropriate to the target environment;
- required GoreeCloud platform-system runtime integrations;
- dependency and artifact integrity checks;
- version/lifecycle correctness; and
- a credible known-good rollback target.

Current Development evidence does not yet satisfy those production gates.

Release preparation

For an eligible release candidate:

1. select the exact merged source SHA;
2. choose a version that does not imply unsupported maturity;
3. run release-specific validation on that exact revision;
4. build artifacts through the controlled GoreeCloud build path;
5. record artifact checksums/digests and required SBOM/signature/provenance evidence;
6. verify package metadata, product identity, icon, supported-platform metadata, and license/provenance;
7. verify migration, upgrade, compatibility, and rollback behavior;
8. prepare release notes with limitations, migrations, and lifecycle status; and
9. preserve the previous known-good source, artifacts, configuration, and recovery point.

Do not publish an artifact merely because `release-please` metadata can calculate a version.

Release publication

When release preparation is independently accepted:

- create an immutable or otherwise controlled version tag resolving to the approved SHA;
- create the matching GitHub release;
- publish only artifacts built from that approved revision;
- retain checksums/digests, SBOMs, signatures, manifests, and provenance required by governance; and
- verify that each published download/package resolves to the expected exact artifact.

Do not rewrite or silently reuse a published release identity.

Deployment authorization

Before production deployment, identify:

- the exact release/artifact;
- the exact target environment;
- the previous known-good runtime and configuration;
- current usable backups/recovery points;
- the rollback or compensating recovery path;
- required secrets through approved secret-management paths;
- storage, TLS, DNS/network, identity, permissions, and dependencies;
- migration order and compatibility;
- monitoring/health/logging sufficient to detect failure; and
- expected maintenance or user impact.

The current bounded SQLite snapshot and clean-target database/database-backed-attachment recovery evidence is useful Development evidence, but it is not yet full-instance recovery. Managed-local files, S3 objects, deployment/user-managed configuration, governed secret/key authority, generations/retention/off-device copies, and full clean-target instance restore remain required where applicable.

Deployment

An authorized deployment must:

1. record the pre-deployment runtime state and rollback target;
2. preserve required mutable data;
3. deploy the exact approved artifact by immutable identity where supported;
4. apply configuration and migrations in the documented order;
5. avoid unrelated changes during the deployment window;
6. capture material errors, warnings, and migration output; and
7. verify the runtime is actually using the intended artifact and configuration.

A successful process start or HTTP 200 alone is not production acceptance.

Post-deployment acceptance

Validate the actual runtime, including the applicable subset of:

- liveness and readiness;
- authentication/authorization and multi-user isolation;

- memo read/write and attachment behavior;
- database integrity and migration state;
- required GoreeCloud integrations;
- privacy/security boundaries and security headers;
- representative GoreeCloud UI (GC UI) and accessibility behavior;
- performance/resource behavior;
- observability and alerting; and
- backup/recovery continuity.

Record the exact deployed release/artifact, environment, checks, results, accepted limitations, and retained rollback target.

Rollback triggers

Rollback or recovery is required when continued operation creates unacceptable risk, including:

- security/privacy failure;
- data corruption or integrity uncertainty;
- failed/incompatible migration;
- authentication/authorization failure;
- critical functional loss;
- severe performance/resource regression;
- required integration failure;
- inability to verify the deployed artifact/configuration; or
- any other release-blocking condition.

Rollback procedure

1. Stop further mutation when continued writes could worsen the failure.
2. Preserve diagnostics and unique post-deployment data that must not be lost.
3. Determine whether artifact/configuration reversal is sufficient or data restoration/compensating recovery is required.
4. Restore the previous known-good application, configuration, and dependent state in the correct order.
5. Restore data only from an approved recovery point when necessary.
6. Re-run the critical acceptance workflows that established the previous state as known-good.
7. Confirm health/monitoring return to expected behavior.
8. Record the rollback as a new controlled event without erasing failed-deployment history.
9. Open corrective engineering work before another promotion attempt.

A one-way data migration can make simple application rollback unsafe; the recovery plan must reflect the real persistent-data boundary.

Emergency fixes

An urgent security, availability, data-integrity, or recovery fix may use an accelerated path, but it still requires:

- a clearly identified baseline and narrow scope;
- exact source and artifact identity;
- the minimum validation necessary to reduce immediate risk;
- a known rollback/recovery path;
- post-deployment validation; and
- completion of deferred ordinary review/documentation as soon as practical.

Evidence and documentation

For every material boundary, preserve the identifiers necessary to reconstruct what happened: repository/branch, PR, exact source SHA, merge SHA, version/tag/release, workflow runs, artifact checksums/digests, migration/schema identity, environment, deployment time, acceptance results, and rollback target.

Use live GitHub for GitHub-native operational state. Update repository documentation and the canonical GoreeCloud Memos task record only from verified evidence.

Production status

Until all applicable gates above are satisfied, GoreeCloud Memos remains Development/nonconformant. Do not claim production deployment, Production Acceptance, Seal, Anchor, or Stable-equivalent maturity from source integration alone.